



JMPM TECNOLOGIA

TECNOLOGIA & DESENVOLVIMENTO

DOCUMENTAÇÃO TÉCNICA

Motor Core GraphQL

Motor Core GraphQL para Protheus — arquitetura, implementação, configuração e operação

Versão **3.1.0**

JMPM Tecnologia

05/09/2026

Sumário

Arquitetura

Mutations

Manual de Implementação

Visão geral da estrutura

Pré-requisitos de ambiente

Passo a passo de deploy

Arquivo de configuração — referência completa

Limitações conhecidas (leia antes de liberar tabelas em produção)

Sem geração automática de chave

Testes

Roteiro de sub-projetos

Documentação Técnica — Este documento consolida a documentação técnica do Motor Core GraphQL para Protheus: arquitetura interna do motor, pipeline de requisição, modelo de mutations e o passo a passo completo de implantação, configuração e operação em ambiente Protheus.

Arquitetura

Pipeline de requisição:

```
GET /graphql --> GQLSERVICE (entrypoints/service.entrypoint.tlpp)
--> GqlLexer + GqlParser (core/lexer.tlpp, core/parser.tlpp)
--> GqlValidator (core/validator.tlpp)
--> GqlExecutor (core/executor.tlpp)
--> GqlQueryBuilder (core/query-builder.tlpp)
--> GqlDictionaryReader (core/dictionary-reader.tlpp)
--> Resposta JSON
```

O schema nunca é escrito à mão: `GqlSchemaProvider` (`core/schema-provider.tlpp`) monta os tipos GraphQL de forma preguiçosa a partir de SX2 (tabelas) e SX3 (campos), e os campos de relacionamento a partir de SX9. Tabelas/campos são filtrados pelo `GqlAccessControl` (`core/access-control.tlpp`) contra as listas de bloqueio do `config/graphql-config.json` antes de qualquer cache.

Verificações de permissão por usuário estão fora do escopo deste sub-projeto; o sub-projeto Auth vai adicioná-las, provavelmente como outro método no `GqlAccessControl`.

Mutations

`GqlMutationExecutor` (`core/mutation-executor.tlpp`) é um pipeline de escrita paralelo ao executor de leitura, compartilhando o mesmo lexer/parser e o mesmo `GqlDictionaryReader` / `GqlQueryBuilder`. Ele escreve via `TCSqlExec()` — **não** via `TCQuery`, que quebra de forma não-capturável com qualquer coisa que não seja `SELECT` neste ambiente (confirmado empiricamente; veja as Restrições Globais do plano para a investigação completa) — e reselectiona a linha afetada através do `GqlExecutor:resolveTableField()` existente, de modo que a moldagem da resposta (aliasas, seleções aninhadas) nunca é duplicada.

Uma tabela é gravável apenas se estiver em `allowMutations` (configuração) E ainda passar pela lista de bloqueio do caminho de leitura — as duas condições se combinam, nenhuma sozinha é suficiente. `GqlInputValidator` verifica obrigatoriedade/tipo/tamanho contra os metadados do SX3 antes de qualquer SQL rodar. Exclusão é sempre lógica (`D_E_L_E_T_ = '*'`), coerente com como toda consulta já filtra leituras.

Campos de entrada desconhecidos ou bloqueados são atualmente ignorados silenciosamente, não rejeitados com erro explícito: `GqlValidator` nunca é invocado no caminho de mutation, então um nome de campo que não faz parte das listas SET/INSERT guiadas pelo dicionário é simplesmente excluído quando essas listas são montadas — nunca chega ao SQL, então isso não é um furo de segurança, mas diverge da semântica normal do GraphQL (um campo desconhecido em um objeto de entrada deveria ser um erro de validação). Fechar isso fica para a validação de argumentos de mutation por campo de um sub-projeto futuro.

Limitação conhecida: Concorrência e atribuição de `R_E_C_N_O_`

Mutations `create` atribuem `R_E_C_N_O_` (a chave primária física da tabela neste caminho de escrita com SQL cru, já que a atribuição automática normal do ISAM/DBAccess é contornada) via uma subconsulta SQL `MAX(R_E_C_N_O_)+1`. Isso **não é seguro sob escrita concorrente** — duas chamadas `create` simultâneas contra a mesma tabela podem entrar em corrida e tentar inserir o mesmo valor de `R_E_C_N_O_`, causando colisão de chave primária.

Tanto uma sequência/IDENTITY real do Postgres quanto uma abordagem de advisory lock na mesma instrução foram investigadas e consideradas indisponíveis/ineficazes neste ambiente. Fechar essa lacuna exige uma migração de banco (adicionar sequências/IDENTITY reais às tabelas de dicionário do Protheus) ou uma stored procedure no servidor — ambos fora do escopo deste sub-projeto.

Garantia de segurança atual: seguro para uso de requisição única e baixa concorrência (ex.: os próprios testes deste sub-projeto). **Não seguro** para tráfego de produção concorrente na mesma tabela.

Veja o cabeçalho Protheus.doc de `buildInsert()` em `custom/backoffice/graphql/core/mutation-builder.tlpp` para a análise técnica completa.

Manual de Implementação

Guia completo para quem vai instalar, configurar e operar este motor GraphQL em um ambiente Protheus.
Público-alvo: administradores de ambiente e desenvolvedores responsáveis pelo deploy.

Visão geral da estrutura

```

custom/backoffice/graphql/
  config/
    graphql-config.json      -- lista de bloqueio, liberação de mutations, paginação
  core/
    lexer.tlpp              -- tokeniza o texto GraphQL recebido
    parser.tlpp              -- monta a árvore de sintaxe (AST) a partir dos tokens
    validator.tlpp           -- valida a AST contra o schema (leitura)
    dictionary-reader.tlpp    -- leitura crua de SX2/SX3/SX9/SIX
    access-control.tlpp       -- aplica a lista de bloqueio de config
    schema-provider.tlpp     -- gera e cacheia o schema GraphQL dinamicamente
    query-builder.tlpp        -- monta o SQL de leitura (SELECT paginado)
    executor.tlpp            -- orquestra leitura: valida, executa, resolve
  relações
    introspection.tlpp       -- responde __schema/__type
    errors.tlpp              -- monta o envelope {"errors": [...]}
    mutation-schema.tlpp     -- expõe create/update/deleteTABLE só para
  tabelas liberadas
    input-validator.tlpp      -- valida input de mutation contra SX3
    mutation-builder.tlpp     -- monta SQL de INSERT/UPDATE/soft-delete
    mutation-executor.tlpp    -- orquestra escrita: valida, escreve,
  reselection
    entrypoints/
      service.entrypoint.tlpp -- ponto de entrada REST único (@Get /
                                graphql)

```

Nenhum tipo GraphQL é escrito à mão. Todo o schema é gerado em tempo de execução a partir do dicionário de dados do Protheus (SX2, SX3, SX9, SIX), com cache em memória por processo do AppServer.

Pré-requisitos de ambiente

- AppServer Protheus com o framework REST habilitado ([HTTPREST] no `appserver.ini` , escutando na porta configurada — 9995 nos exemplos deste manual).
- Backend de banco de dados: o motor foi desenvolvido e testado contra **PostgreSQL**. A escrita usa SQL bruto via `TCSqlExec()` — não foi validada contra outros bancos suportados pelo Protheus (SQL Server, Oracle, DB2).
- Ferramental de compilação de fontes `.tlpp` para o seu ambiente (`appsrvlinux -compile` ou equivalente).

Passo a passo de deploy

1. Compilar os fontes

Compile, na ordem de dependência abaixo, todos os arquivos de `custom/backoffice/graphql/core/` seguidos de `custom/backoffice/graphql/entrypoints/service.entrypoint.tlpp`:

```
config.tlpp
access-control.tlpp
dictionary-reader.tlpp
schema-provider.tlpp
lexer.tlpp
parser.tlpp
validator.tlpp
query-builder.tlpp
executor.tlpp
errors.tlpp
introspection.tlpp
mutation-schema.tlpp
input-validator.tlpp
mutation-builder.tlpp
mutation-executor.tlpp
entrypoints/service.entrypoint.tlpp
```

Compilar o conjunto inteiro junto (não só o arquivo alterado) evita erros de "classe inválida" quando o ambiente de compilação não tem as classes de uma sessão anterior em cache.

2. Implantar o RPO

Copie o RPO compilado para o AppServer de destino e reinicie o processo, seguindo o processo padrão do seu ambiente.

3. Implantar o arquivo de configuração (passo manual obrigatório)

Este é o passo mais fácil de esquecer, e o mais silencioso quando esquecido.

`custom/backoffice/graphql/config/graphql-config.json` **não** é um fonte `.tlpp` — ele nunca entra no RPO compilado. Ele precisa ser copiado manualmente para o sistema de arquivos do servidor, no caminho:

```
<RootPath>/custom/backoffice/graphql/config/graphql-config.json
```

Onde `<RootPath>` é o valor de `[P12] RootPath=` no `appserver.ini` do ambiente de destino — **não** o `SourcePath`, e não o diretório de trabalho do processo do AppServer. Isso foi confirmado por teste direto contra um servidor real: `MemoRead()` (a função que a classe `GqlConfig` usa para ler o JSON) resolve caminhos relativos contra o `RootPath`, ignorando as outras três localizações candidatas testadas.

Consequência de esquecer este passo: `GqlConfig.new()` não encontra o arquivo, cai silenciosamente nos valores padrão embutidos (lista de bloqueio vazia, nenhuma tabela liberada para mutation), e **todas as tabelas do dicionário ficam visíveis e legíveis via GraphQL** — incluindo tabelas sensíveis como `SRH*`

(Recursos Humanos), a menos que o `graphql-config.json` real esteja de fato no lugar certo. Sempre confirme após o deploy:

```
curl "http://localhost:9995/rest/graphql" | grep -o '"SRA"'
```

Não deve haver saída (assumindo `SRA` na sua lista de bloqueio) — se aparecer, o arquivo de configuração não foi encontrado pelo servidor.

4. Confirmar que o serviço está no ar

```
curl "http://localhost:9995/rest/graphql"
```

Deve retornar a lista de tipos disponíveis (`__schema.types`).

Arquivo de configuração — referência completa

`custom/backoffice/graphql/config/graphql-config.json` :

```
{
  "denyTables": ["SRH*", "SRA", "SR5"],
  "denyFields": ["*SENHA*", "*_PASSWORD*"],
  "allowMutations": ["SA1"],
  "pagination": { "defaultPageSize": 20, "maxPageSize": 200 },
  "schemaCacheTtlSeconds": 3600
}
```

Chave	Tipo	Significado
<code>denyTables</code>	array de strings com curinga	Aliases de tabela nunca expostos, ex. <code>"SRH*"</code>
<code>denyFields</code>	array de strings com curinga	Nomes de campo nunca expostos, ex. <code>"*SENHA*"</code>
<code>allowMutations</code>	array de aliases exatos (sem curinga)	Tabelas que aceitam <code>createTABLE</code> / <code>updateTABLE</code> / <code>deleteTABLE</code> ; vazio por padrão (tudo somente leitura)
<code>pagination.defaultPageSize</code>	número	<code>limit</code> usado quando a consulta não informa um
<code>pagination.maxPageSize</code>	número	Teto rígido de <code>limit</code> , mesmo que a consulta peça mais
<code>schemaCacheTtlSeconds</code>	número	Tempo que o tipo gerado de uma tabela fica em cache antes de ser reconstruído automaticamente

Curingas (`*`) funcionam como "qualquer sequência de caracteres" e aplicam-se apenas a `denyTables` / `denyFields` . `allowMutations` exige o alias exato — não aceita curinga, por design: liberar escrita é uma decisão por tabela, não um padrão amplo.

Tabelas/campos bloqueados nunca aparecem em introspecção nem em resultado de consulta, independentemente da forma da query. Uma tabela só é gravável se estiver **ao mesmo tempo** em `allowMutations` e ausente de `denyTables` — as duas condições precisam valer juntas.

Limitações conhecidas (leia antes de liberar tabelas em produção)

Concorrência na criação de registros (`R_E_C_N_O_`)

Mutations de `create` escrevem via SQL bruto, o que faz o caminho normal de atribuição automática de `R_E_C_N_O_` (a chave física de registro do Protheus) pelo ISAM/DBAccess ser contornado. O motor calcula esse valor via uma subconsulta `MAX(R_E_C_N_O_) + 1` .

Isso não é seguro sob escrita concorrente: duas chamadas `create` simultâneas contra a mesma tabela podem calcular o mesmo próximo valor antes de qualquer uma delas confirmar a transação, causando colisão de chave primária. Foram investigadas e descartadas duas alternativas (uma sequência real do PostgreSQL — não existe para as tabelas do dicionário Protheus neste tipo de ambiente — e um lock consultivo do PostgreSQL dentro da mesma instrução — testado e confirmado ineficaz sob o nível de isolamento `READ COMMITTED`). Resolver isso de verdade exige uma migração de banco (sequência/`IDENTITY` real nas tabelas do dicionário) ou uma stored procedure no servidor — ambos fora do escopo atual.

Garantia de segurança atual: seguro para uso de requisição única ou baixa concorrência. **Não seguro** para tráfego de produção concorrente contra a mesma tabela. Veja o cabeçalho Protheus.doc de `buildInsert()` em `mutation-builder.tlpp` para a análise técnica completa.

Casamento de linha em update/delete depende do índice SIX

`update / delete` identificam a linha certa usando a chave de ordem 1 do índice `SIX` da tabela. Em ambientes onde `RetSqlName("SIX")` não resolve (comum em instalações de desenvolvimento/teste), o motor tenta um segundo caminho, derivando o nome físico da tabela `SIX` a partir do sufixo de empresa já usado por `RetSqlName("SX3")` — e só recorre ao primeiro campo escalar da tabela como último recurso. Se nenhuma das duas formas encontrar a chave real de order 1, o casamento cai para um único campo, o que pode fazer `update / delete` afetar a linha errada em tabelas com mais de uma linha ativa por filial.

Antes de liberar uma nova tabela em `allowMutations` , confirme que sua chave de ordem 1 é corretamente identificada: crie dois registros de teste na mesma filial, diferindo apenas no campo que deveria ser único (ex. o código), faça um `update` em apenas um deles e confirme que o outro permanece inalterado:

```
mutation { createTABELA(input: {COD: "TESTE1", ...}) { COD } }
mutation { createTABELA(input: {COD: "TESTE2", ...}) { COD } }
mutation { updateTABELA(input: {COD: "TESTE1", ..., NOME: "Alterado"}) { NOME } }
```


Se o registro `TESTE2` também aparecer alterado (ou se `updateTABELA` com uma chave inexistente retornar sucesso em vez de `"Row not found for update"`), a tabela não deve ser liberada para mutation até o índice `SIX` correspondente ser corrigido no ambiente.

Sem validações automáticas do Protheus no caminho de escrita

Mutations não passam pelo `FWFormModel /MVC` — nenhum trigger `SX7`, regra de negócio de rotina, ou fórmula `X3_VALID` é executada. A única validação aplicada é a que o motor faz explicitamente: campo obrigatório (`X3_OBRIGAT`), tipo e tamanho máximo (`X3_TAMANHO`), a partir do `SX3`.

Campos desconhecidos no `input` são ignorados silenciosamente

Se o cliente enviar no `input` de uma mutation um campo que não existe na tabela (ou que está bloqueado por `denyFields`), o motor simplesmente o ignora — não gera erro. Isso não é uma falha de segurança (o campo nunca chega a ser escrito, já que as listas de campos usadas na escrita vêm sempre do dicionário, nunca diretamente do `input` do cliente), mas diverge da semântica usual do GraphQL, onde um campo desconhecido em um objeto de entrada normalmente gera erro de validação. Fica para um sub-projeto futuro (Auth) fechar essa lacuna.

Sem geração automática de chave

O cliente sempre precisa enviar o(s) campo(s) de chave no `input`, inclusive em `create` — o motor não gera código automaticamente via `SX5 (GetSxeNum())`. Quem precisar de numeração automática deve gerar a chave antes de chamar a mutation.

Testes

Testes end-to-end em Python (framework TIR) ficam em `tests/tir/`. Execute com `pytest tests/tir/ -v` contra um AppServer Protheus real com este RPO implantado. Não há suíte de testes unitários AdvPL/TLPP neste projeto — a verificação em desenvolvimento foi feita via `curl` ao vivo contra um servidor real, e os testes TIR documentam o comportamento esperado para verificação contínua.

Roteiro de sub-projetos

Este motor faz parte de um roteiro maior em 6 sub-projetos:

1. **Core Engine** (leitura) — concluído
2. **Mutations** (escrita) — concluído, este manual
3. **Auth** — autenticação/autorização real por usuário
4. **Field Hooks** — pontos de extensão por campo
5. **SDK Generator** — geração de contratos AdvPL a partir do schema
6. **Console PO-UI** — interface administrativa

Cada sub-projeto tem sua própria especificação em `docs/superpowers/specs/`.